

广义五子棋游戏与对抗搜索 (Assignment - 1)

对应课程内容：第 4 章 对抗搜索与博弈 (Adversarial Search & Games)

发布时间：2026-09-28 | 截止时间：2026-10-11

作业简介

本次作业要求每位同学**独立实现一个广义五子棋 (Gomoku) 游戏**：在 $N \times N$ 的棋盘上，双方轮流落子，先连成 K 子者获胜（即 (m,n,k)-Game；标准五子棋取 $N = 15, K = 5$ ，也是本作业的默认配置）。棋盘大小 N 与连珠数 K 都是**可配置参数**，你的程序与 AI 必须自适应任意合法组合，不得写死。

- 你的游戏需要支持三种对战模式：**人人对战、人机对战、机机对战**。
- 你的 AI 必须遵循本讲义定义的**统一对战协议**，并在每步的**限定时间内输出合法落子**。这是本作业的硬性要求：课程裁判程序将按统一协议与不同 (N, K) 配置测试你的 AI；不同同学的 AI 也可以用同一协议相互对战（鼓励，但对抗结果不作为评分依据）。

本作业是第 4 章内容的直接实践：博弈的形式化、MiniMax 搜索、Alpha-Beta 剪枝、资源限制与深度截断、评估函数，以及可选的蒙特卡罗树搜索 (MCTS)，都会在实现中真正派上用场。

一、游戏规则

项目	规定
棋盘	$N \times N$ 方格棋盘， N 由裁判/配置指定（默认 15，本作业中 $5 \leq N \leq 15$ ）
连珠	K 子连珠获胜， K 由裁判/配置指定（ $3 \leq K \leq N$ ；默认示例 $K = 5$ ，也可以是 6、7 等）
行棋	黑棋（编号 1）先行，双方轮流在任意空位落子，落子后不可移动
胜负	横、竖、两条斜线任一方向上连成 K 子及以上者获胜（长连同样判胜，无禁手）
和棋	棋盘下满仍无 K 连，判为和棋 (Draw)

注意： N 与 K 的不同组合会显著改变博弈性质（例如 K 越大，棋局越长、越难速胜）。评赛时会使用**多组不同的 (N, K) 配置**，你的 AI 必须全部自适应。

二、统一对战协议 (Gomoku AI Protocol v1)

2.1 棋盘表示

- 棋盘是一个 $N \times N$ 的二维列表（列表的列表）， N 通过构造函数传入。
- 0 表示空位，1 表示黑棋，2 表示白棋。
- 坐标使用 (row, col) 元组，**均从 0 开始编号**，(0, 0) 为棋盘左上角。

2.2 提交文件

每位同学提交一个名为 `gomoku_ai.py` 的文件，其中必须包含一个名为 `GomokuAI` 的类，并实现以下接口：

```

class GomokuAI:
    """统一对战协议 v1 — 课程裁判程序将按此接口加载并调用你的 AI。"""

    def __init__(self, player_id: int, board_size: int, win_length: int):
        """
        构造函数。每局比赛开始时，裁判会为新的一局重新创建一个 GomokuAI 实例。
        :param player_id: 你在本局中的棋色，1 = 黑棋（先手），2 = 白棋（后手）。
        :param board_size: 棋盘边长 N（本作业中  $5 \leq N \leq 15$ ，但不排除其他取值）。
        :param win_length: 连珠数 K ( $3 \leq K \leq N$ )。
        """
        self.player_id = player_id
        self.board_size = board_size
        self.win_length = win_length
        # 在这里初始化你的内部状态（例如空搜索树、统计变量等）

    def get_move(self, board, last_opponent_move, time_limit):
        """
        轮到你落子时，裁判会调用本函数。
        :param board: 当前棋盘，board_size x board_size 的二维列表；
            0=空位，1=黑棋，2=白棋。视为只读数据（裁判传入的是副本）。
        :param last_opponent_move: 对手刚下的落子坐标 (row, col)；
            若对手尚未落过子（你是先手第一回合），为 None。
        :param time_limit: 本次落子的时限，单位为秒（当前赛制为 5.0 秒）。
        :return: 你决定的落子坐标 (row, col)。
        """
        raise NotImplementedError

```

此外，文件中可以（可选）定义一个模块级变量作为裁判输出中的显示名：

```
NAME = "your-ai-name" # 缺省时使用文件所在目录名
```

提交后，助教将直接与你课程 GitHub 仓库的 `assignment1/gomoku_ai.py` 加载该文件（见“八、提交内容与方式”），因此**文件名、类名与接口签名不可更改**。

2.3 裁判判定规则

裁判程序负责棋局管理、合法性检查与计时。对局中一旦出现下列任一情况，**该 AI 直接判负**：

1. **超时**：`get_move` 超过时限仍未返回（按墙钟时间计）。
2. **非法落子**：返回坐标越界、指向已占用格，或返回类型不是 `(int, int)` 元组。
3. **运行异常**：`get_move` 抛出未捕获异常，或构造函数失败。

除上述情况外：

- 落子合法则按规则落子、检查 K 连；一方连珠即胜，棋盘下满则和棋。
- 裁判可在任意 (N, K) 配置下运行对局，并可选择任意对手（随机 AI、参考 AI、或其他同学的 AI）。

硬性要求：在限定时间内按协议返回合法落子，是对 AI 的最低要求。无论你的搜索算法多强，超时、异常或非法落子都会导致判负——请务必做好时间管理与输入校验。

2.4 参考实现

课程提供一个最简单的参考 AI `random_ai.py`（随机合法落子），用于功能测试和实力基线。你的 AI 应当能够稳定战胜它：

```
# random_ai.py — 参考：随机合法落子
import random

class GomokuAI:
    def __init__(self, player_id, board_size, win_length):
        self.player_id = player_id
        self.board_size = board_size
        self.win_length = win_length

    def get_move(self, board, last_opponent_move, time_limit):
        empties = [(r, c) for r in range(self.board_size)
                    for c in range(self.board_size) if board[r][c] == 0]
        return random.choice(empties)
```

2.5 协议注意事项

- **不得硬编码 N 与 K** ：棋盘大小与连珠数只能来自构造参数；任何写死尺寸或"五连"的代码都无法通过多配置测试。
- **不要原地修改传入的 `board`**：裁判传入的是副本，修改它不会影响真实棋局，但依赖这一行为的代码在自建引擎中会出错。
- **自己管理时限**：`time_limit` 参数已经给出，请在搜索中做好深度截断/迭代加深等时间管理，不要"顶格"使用全部时限。
- **每局一个新实例**：不要假设构造函数只被调用一次；跨局记忆不会带来好处（对手每局可能不同）。
- **协议兼容性是硬性要求**：`gomoku_ai.py` 必须能在裁判程序中与任意同学的 AI、任意 (N, K) 配置正常对战。

三、游戏程序要求

除统一协议的 AI 文件外，每位同学需要实现自己的广义五子棋游戏程序，支持三种模式。界面形式自选（GUI 或 CLI 均可）。建议的文件结构（可自由调整，评分将综合考察你提交的全部代码与实验报告）：

```
assignment1/
├── main.py           # 游戏主程序入口（GUI 或统一入口）
├── main_cli.py       # 命令行入口（若采用 CLI 界面）
├── gomoku_game.py    # 棋局逻辑：落子、K 连判定、和棋判定等
├── gui.py            # 图形界面绘制（若采用 GUI）
├── gomoku_ai.py      # 你的 AI（统一协议，评赛用）
├── random_ai.py      # 随机 AI（参考实现，可作测试对手）
└── requirements.txt  # 项目依赖
```

三种模式的说明：

1. **人人对战**：两名玩家交替点击/输入坐标落子，程序判定胜负。
2. **人机对战**：玩家执一方，你的 AI 执另一方，由你的游戏引擎调用你的 `GomokuAI`。
3. **机机对战**：你的游戏引擎加载两个符合统一协议的 AI（可以是自己的两份、或自己的 AI 对战同学的 AI 文件），自动完成整局。

游戏程序本身也必须支持配置 N 与 K （例如命令行参数 `--board 15 --win 5`，或 GUI 中的设置项），并能正确处理不同组合下的胜负与和棋判定。

四、AI 算法要求

- **基础要求（必须）**：使用课堂介绍的对抗搜索框架，即 **MiniMax 搜索 + Alpha-Beta 剪枝**，配合：

- 深度截断与**评估函数 (Evaluation Function)** (例如基于连子数量、"K-1 连活棋"等棋型的启发式打分, 评估需以 K 为参数自适应);
- 合理的**候选着法生成** (例如只考虑已有棋子附近的空位, 而非全部空位);
- 在每步时限 (默认 5 秒) 内稳定返回合法落子。
- **进阶方向 (加分)**: 迭代加深 (Iterative Deepening)、着法排序 (Move Ordering)、蒙特卡罗树搜索 (MCTS) 及 UCT、更好的模型评估等。报告中应说明原理并给出对比实验。
- **允许**: Python 标准库、numpy、课程讲义与教材中的伪代码。
- **禁止**:
 - 调用任何外部五子棋引擎、在线服务或网络资源;
 - 内嵌针对特定对手 (识别对手 ID/名字) 的针对性策略;
 - 以查表方式内嵌预计算好的固定应答 (核心决策必须来自运行时搜索与评估; 首手下中心这类一般性启发规则不受限)。

五、课程提供的裁判工具

课程随本讲义发布裁判程序 `arena.py` 与随机 AI 参考 `random_ai.py` (两个文件与本讲义一起下载, 无目录结构), 用于协议测试与 (可选的) 相互对战:

```
# 你的 AI 对阵随机 AI (15x15, 五连, 每步 5 秒, 默认两局交换先后手)
python arena.py assignment1/gomoku_ai.py random_ai.py --board 15 --win 5

# 与同学的 AI 对战 (9x9, 四连, 每步 1 秒, 只打 1 局, 安静模式)
python arena.py assignment1/gomoku_ai.py classmate/gomoku_ai.py --board 9 --win 4 --time 1.0 --games 1 --quiet
```

- `arena.py`: 加载两份 AI 文件进行比赛, 逐步打印棋盘与落子, 最后输出比分。完整参数: `--board N` (棋盘边长, 默认 15)、`--win K` (连珠数, 默认 5)、`--time T` (每步时限, 秒, 默认 5.0)、`--games G` (对局数, 交换先后手, 默认 2)、`--quiet` (只看结果)。
- 超时、异常、非法落子会被裁判立即判负, 并在结果中注明原因。
- **评赛方式**: 助教将直接从各位的课程 GitHub 仓库加载 `assignment1/gomoku_ai.py`, 用统一裁判、以评赛前公布的若干组 (N, K) 配置进行合规性测试 (对手可能是随机 AI、参考 AI 或其他同学的 AI)。对抗胜负不作为评分依据, 协议合规才是。

六、评分标准

本作业最鼓励大家做好的是两件事: **一是把 AI 的功能真正实现好**——在统一协议下稳定运行, 用课堂所学的对抗搜索下出有质量的棋; **二是把报告写得清晰**——让读者能看懂你的设计思路、算法与实验结果。同时, 规定的三种基本对战模式 (人人、人机、机机) 是必须完成的基础要求, 在此之上, AI 与报告的质量决定成绩高低。

七、报告要求

报告以 Markdown 撰写, 保存为 `assignment1/report.md` (可另附 `report.pdf`)。报告的核心是**清晰**: 设计思路、算法与实验结果都要让读者看得懂。内容包括:

1. **总体设计**: 游戏程序结构、三种模式的实现方式、 N 与 K 的配置机制。
2. **AI 算法说明**: 搜索框架 (伪代码)、评估函数设计与直观解释、剪枝/候选着法/时限管理等优化, 以及 AI 如何自适应不同的 N 与 K 。
3. **实验评估**: 至少包含——
 - 你的 AI 对 `random_ai` 的胜率 (各 20 局以上, 报告平均每步思考时间);
 - 你的 AI 自我对战 (先后手各若干局) 的胜负与平均对局长度;
 - 至少两组不同 (N, K) 配置下的表现对比;

- （若实现）不同搜索深度或不同评估函数之间的对比。

4. 讨论与反思：遇到的困难、评估函数的不足、若继续改进会做什么。

八、提交内容与方式

本学期全部通过你与教师共享的 **GitHub 课程仓库** 提交。在你的仓库中建立 `assignment1/` 目录：

```
assignment1/
├── gomoku_ai.py      # 统一协议 AI（路径与类名不可更改，评赛直接从此加载）
├── main.py           # 游戏代码（main_cli.py / gomoku_game.py / gui.py 等）
├── random_ai.py      # （可选）随机 AI，便于自测
├── requirements.txt
└── report.md         # 实验报告（Markdown，可另附 report.pdf）
```

- **截止判定**：以截止时间前仓库中 `assignment1/` 目录的最后一次 push 为准。
- 评赛使用的 `gomoku_ai.py` 以你仓库中截止时的版本为准，**截止后不再接受补交修改**。
- 请勿将无关的大文件（数据集、模型权重等）提交进仓库。

九、学术诚信

- 本作业须独立完成；与他人讨论思路可以，但代码不得共享或抄袭。查重工具将检查所有提交的 `gomoku_ai.py`。
- 引用任何外部资料（含教材、博客、开源代码片段）须在报告中注明出处。
- 违反学术诚信者按学校相关规定处理。

十、时间安排

节点	时间
作业发布	2026-09-28
提交截止	2026-10-11